

Срок сдачи: четверг, 16 апреля 2026, 23:59

1. ЦЕЛЬ РАБОТЫ

Освоение принципов асинхронного программирования в JavaScript, работы с Fetch API, форматами JSON и клиентским хранением данных. Формирование навыков взаимодействия с внешними API и организации локального хранения состояния приложения.

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- Основы JavaScript (переменные, функции, объекты, массивы)
- Работу с DOM и обработку событий (из ЛР №5)
- Понятие асинхронности и цикл событий в JavaScript
- Базовые принципы HTTP-запросов
- Структуры данных JSON (**JSON** — формат данных, который возвращают большинство API).
- Основы безопасности веб-приложений

Дополнительно:

- **OAuth 2.0, API-ключи** — для работы с API, требующими авторизации (например, Google API).
- **Безопасность на клиенте** — понимание ограничений хранения чувствительных данных

3. ЗАДАЧИ РАБОТЫ

- Освоить работу с Promise, async/await и Fetch API (**REST API** и **Fetch/Axios** для отправки запросов к API).
- Научиться выполнять HTTP-запросы к внешним API
- Реализовать обработку данных в формате JSON
- Организовать клиентское хранение данных с использованием LocalStorage/SessionStorage
- Реализовать обработку ошибок при асинхронных операциях
- Интегрировать полученные данные в существующий интерфейс
- Освоить основы безопасности при работе с API

4. ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

Выбор стороннего сервиса и настройка API

Шаг 1: Выбор стороннего сервиса

Выберите один из предложенных сервисов для интеграции согласно вашему варианту проекта:

```
// Примеры популярных API для разных типов проектов
const API_OPTIONS = {
  // Медиа и контент
  news: 'NewsAPI, Guardian API',
  weather: 'OpenWeatherMap, WeatherAPI',
  movies: 'OMDb API, The Movie Database',
  books: 'Google Books API, Open Library',
  // Социальные и развлекательные
  social: 'JSONPlaceholder (для тестирования)',
  music: 'Spotify Web API, Last.fm API',
  images: 'Unsplash API, Pexels API',
  // Сервисы и утилиты
  maps: 'Google Maps API, OpenStreetMap',
  finance: 'CoinGecko API, Alpha Vantage',
  education: 'Khan Academy API, Wikipedia API'
};
```

1. Шаг 2: Регистрация и получение ключей

- Зарегистрируйтесь на платформе выбранного сервиса
- Получите API-ключ или настройте OAuth 2.0 аутентификацию
- Изучите документацию API (доступные endpoints, параметры, лимиты)

Шаг 3: Настройка проекта

```
# Создайте файл .env для хранения конфиденциальных данных
echo "API_KEY=your_actual_api_key_here" > .env
echo "API_BASE_URL=https://api.example.com" >> .env
```

Часть 1. Подготовка проекта и настройка структуры

1. Перейдите в рабочую ветку:

```
git checkout main
git pull origin main
git checkout -b feature/async-data-storage
```

2. Создайте структуру файлов для работы с данными:

Интерфейс должен включать:

- Форму для ввода данных (например, для поиска города, места на карте или видео).
- Панель для отображения полученных данных (например, карта, погодные условия, результаты поиска).

Подключает JavaScript, где будет реализована логика взаимодействия с API.

- В папке js создайте структуру файлов для работы с данными:

```
js/
├── api/
│   ├── apiService.js
│   └── config.js
├── storage/
│   ├── localStorage.js
│   └── sessionStorage.js
├── utils/
│   └── dataParser.js
└── script.js
```

Часть 2. Реализация API сервиса

Реализация Fetch запросов (или Axios)

1. Изучает документацию выбранного API и определяет эндпоинты для выполнения запросов.
2. Реализует функцию для отправки **GET** или **POST** запросов с помощью **Fetch** или **Axios**:
 - Пример с использованием Fetch

```
fetch('https://api.example.com/data', {
  method: 'GET',
  headers: {
    'Authorization': `Bearer ${process.env.API_KEY}`
  }
})
.then(response => response.json())
.then(data => {
  console.log(data);
  // Обработка данных
})
.catch(error => console.error('Error:', error));
```

Базовые HTTP-запросы (js/api/apiService.js):

```
class ApiService {
  constructor(baseUrl, apiKey = null) {
    this.baseUrl = baseUrl;
    this.apiKey = apiKey;
  }

  // GET запрос с авторизацией
  async get(endpoint, params = {}) {
    try {
      // Добавляем API ключ к параметрам если требуется
      const queryParams = new URLSearchParams({
        ...params,
        ...(this.apiKey && { api_key: this.apiKey }),
      }).toString();

      const url = `${this.baseUrl}${endpoint}${queryParams}`;
      const response = await fetch(url, {
        method: 'GET',
        headers: {
          'Content-Type': 'application/json',
        },
      });

      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }

      const data = await response.json();
      return data;
    } catch (error) {
      console.error('GET request failed:', error);
      throw error;
    }
  }
}
```

```

// POST запрос
async post(endpoint, data) {
  try {
    const response = await fetch(`${this.baseURL}${endpoint}`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(data),
    });

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    return await response.json();
  } catch (error) {
    console.error('POST request failed:', error);
    throw error;
  }
}

// Универсальный метод для различных API
async fetchFromAPI(endpoint, options = {}) {
  const defaultOptions = {
    method: 'GET',
    headers: {
      'Content-Type': 'application/json',
    },
  };

  try {
    const response = await fetch(`${this.baseURL}${endpoint}`, {
      ...defaultOptions,
      ...options,
    });

    if (!response.ok) {
      throw new Error(
        `API request failed: ${response.status} ${response.statusText}`
      );
    }

    return await response.json();
  } catch (error) {
    console.error('API request failed:', error);
    throw error;
  }
}

export default ApiService;

```

Конфигурация API (js/api/config.js):

```

// Конфигурация для различных API
export const API_CONFIG = {
  // Примеры популярных API
  weather: {
    url: 'https://api.openweathermap.org/data/2.5',
    apiKey: process.env.WEATHER_API_KEY,
    endpoints: {
      current: '/weather',
      forecast: '/forecast',
    },
  },
  news: {
    url: 'https://newsapi.org/v2',
    apiKey: process.env.NEWS_API_KEY,
    endpoints: {
      headlines: '/top-headlines',
      everything: '/everything',
    },
  },
  unsplash: {
    url: 'https://api.unsplash.com',
    apiKey: process.env.UNSPLASH_API_KEY,
    endpoints: {
      photos: '/photos',
      search: '/search/photos',
    },
  },
  // Добавьте конфигурацию для вашего выбранного API
  custom: {
    url: 'https://api.yourservice.com/v1',
    apiKey: process.env.YOUR_API_KEY,
    endpoints: {
      // Определите endpoints согласно документации
    },
  },
};

// Фолбэк данные для демонстрации при недоступности API
export const FALLBACK_DATA = {
  weather: {
    temp: 20,
    description: 'Ясно',
    city: 'Минск',
  },
  news: [
    {
      title: 'Пример новости',
      description: 'Описание новости',
      publishedAt: new Date().toISOString(),
    },
  ],
};

```

Часть 3. Работа с клиентским хранилищем

LocalStorage сервис (js/storage/localStorage.js):

```

class LocalStorageService {
  constructor() {
    this.storage = window.localStorage;
    this.initializeStorage();
  }

  initializeStorage() {
    // Инициализация необходимых ключей
    if (!this.get('app_settings')) {
      this.set('app_settings', {
        theme: 'light',
        language: 'ru',
        cacheDuration: 3600000, // 1 час в миллисекундах
      });
    }
  }

  // Сохранение данных с временной меткой
  set(key, value) {
    try {
      const item = {
        value: value,
        timestamp: new Date().getTime(),
      };
      const serializedValue = JSON.stringify(item);
      this.storage.setItem(key, serializedValue);
      return true;
    } catch (error) {
      console.error('Error saving to localStorage:', error);
      return false;
    }
  }

  // Получение данных с проверкой актуальности
  get(key, defaultValue = null, maxAge = null) {
    try {
      const item = this.storage.getItem(key);
      if (!item) return defaultValue;

      const parsedItem = JSON.parse(item);

      // Проверка актуальности данных
      if (maxAge && new Date().getTime() - parsedItem.timestamp > maxAge) {
        this.remove(key);
        return defaultValue;
      }

      return parsedItem.value;
    } catch (error) {
      console.error('Error reading from localStorage:', error);
      return defaultValue;
    }
  }

  // Удаление данных
  remove(key) {
    try {
      this.storage.removeItem(key);
      return true;
    } catch (error) {
      console.error('Error removing from localStorage:', error);
      return false;
    }
  }

  // Очистка устаревших данных
  clearExpired() {
    const settings = this.get('app_settings');
    const cacheDuration = settings?.cacheDuration || 3600000;
    const now = new Date().getTime();

    Object.keys(this.storage).forEach((key) => {
      if (key !== 'app_settings') {
        const item = this.storage.getItem(key);
        if (item) {
          try {
            const parsedItem = JSON.parse(item);
            if (now - parsedItem.timestamp > cacheDuration) {
              this.remove(key);
            }
          } catch (error) {
            // Невалидные данные - удаляем
            this.remove(key);
          }
        }
      }
    });
  }
}

```

```

+ // Получение всех ключей
+ getAllKeys() {
+   return Object.keys(this.storage);
+ }

+ // Проверка существования актуальных данных
+ isValid(key, maxAge = null) {
+   return this.get(key, null, maxAge) !== null;
+ }
+ }

```

```
export default LocalStorageService;
```

Часть 4. Интеграция с UI и обработка состояний

Утилиты для работы с данными (js/utls/dataParser.js)

```

/**
 * Утилиты для обработки и преобразования данных
 */

// Парсинг JSON с обработкой ошибок
export const parseJSON = (jsonString) => {
  try {
    return JSON.parse(jsonString);
  } catch (error) {
    console.error('JSON parsing error:', error);
    return null;
  }
};

// Форматирование даты
export const formatDate = (dateString) => {
  try {
    const date = new Date(dateString);
    if (isNaN(date.getTime())) {
      return 'Неверная дата';
    }
    return date.toLocaleDateString('ru-RU', {
      year: 'numeric',
      month: 'long',
      day: 'numeric',
      hour: '2-digit',
      minute: '2-digit',
    });
  } catch (error) {
    console.error('Date formatting error:', error);
    return 'Ошибка даты';
  }
};

// Ограничение текста с учетом слов
export const truncateText = (text, maxLength, useWordBoundary = true) => {
  if (!text || text.length <= maxLength) return text;

  let truncated = text.substr(0, maxLength);

  if (useWordBoundary) {
    truncated = truncated.substr(
      0,
      Math.min(truncated.length, truncated.lastIndexOf(' ')),
    );
  }

  return truncated + '...';
};

```

```
// Создание DOM элемента из данных и шаблона
export const createElementFromData = (data, template) => {
  try {
    let html = template;

    // Замена плейсхолдеров на реальные данные
    Object.keys(data).forEach((key) => {
      const placeholder = `{{${key}}}`;
      const value = data[key] || '';
      html = html.replace(new RegExp(placeholder, 'g'), value);
    });

    const templateElement = document.createElement('template');
    templateElement.innerHTML = html.trim();
    return templateElement.content.firstChild;
  } catch (error) {
    console.error('Error creating element from template:', error);
    return document.createElement('div');
  }
};

// Форматирование чисел (цен, рейтингов и т.д.)
export const formatNumber = (number, options = {}) => {
  const defaults = {
    minimumFractionDigits: 0,
    maximumFractionDigits: 2,
  };

  return new Intl.NumberFormat('ru-RU', { ...defaults, ...options }).format(
    number,
  );
};

// Форматирование валюты
export const formatCurrency = (amount, currency = 'RUB') => {
  return new Intl.NumberFormat('ru-RU', {
    style: 'currency',
    currency: currency,
  }).format(amount);
};

// Создание строки запроса из объекта параметров
export const buildQueryString = (params) => {
  const searchParams = new URLSearchParams();

  Object.keys(params).forEach((key) => {
    if (params[key] !== null && params[key] !== undefined) {
      searchParams.append(key, params[key]);
    }
  });

  return searchParams.toString();
};
```



```

// Безопасное получение вложенного свойства объекта
export const getNestedValue = (obj, path, defaultValue = null) => {
  try {
    const value = path.split('.').reduce((current, key) => {
      return current && current[key] !== undefined ? current[key] : undefined;
    }, obj);

    return value !== undefined ? value : defaultValue;
  } catch (error) {
    console.error('Error getting nested value:', error);
    return defaultValue;
  }
};

// Фильтрация массива объектов по нескольким критериям
export const filterData = (data, filters) => {
  return data.filter((item) => {
    return Object.keys(filters).every((key) => {
      const filterValue = filters[key];
      const itemValue = item[key];

      if (
        filterValue === '' ||
        filterValue === null ||
        filterValue === undefined
      ) {
        return true;
      }

      if (typeof filterValue === 'string') {
        return itemValue
          ?.toString()
          .toLowerCase()
          .includes(filterValue.toLowerCase());
      }

      return itemValue === filterValue;
    });
  });
};

// Сортировка массива объектов
export const sortData = (data, key, direction = 'asc') => {
  return [...data].sort((a, b) => {
    let aValue = a[key];
    let bValue = b[key];

    // Приведение к строке для сравнения, если значения разные
    if (typeof aValue === 'string' && typeof bValue === 'string') {
      aValue = aValue.toLowerCase();
      bValue = bValue.toLowerCase();
    }

    if (aValue < bValue) return direction === 'asc' ? -1 : 1;
    if (aValue > bValue) return direction === 'asc' ? 1 : -1;
    return 0;
  });
};

// Генерация уникального ID
export const generateId = (prefix = '') => {
  return (
    prefix + Date.now().toString(36) + Math.random().toString(36).substr(2)
  );
};

// Проверка типа данных
export const isObject = (value) => {
  return value && typeof value === 'object' && !Array.isArray(value);
};

export const isArray = (value) => {
  return Array.isArray(value);
};

export const isEmpty = (value) => {
  if (value === null || value === undefined) return true;
  if (typeof value === 'string') return value.trim() === '';
  if (Array.isArray(value)) return value.length === 0;
  if (typeof value === 'object') return Object.keys(value).length === 0;
  return false;
};

```

Примеры использования этих утилит в основном коде:

```
// В script.js - использование createElementFromData
createDataElement(item) {
  ...const template = `
  ...<div class="data-item card">
  ...<h3 class="data-item__title">{{title}}</h3>
  ...<p class="data-item__description">{{description}}</p>
  ...<div class="data-item__meta">
  ...<span class="data-item__date">{{formattedDate}}</span>
  ...<span class="data-item__author">{{author}}</span>
  ...<button class="btn btn--secondary" data-id="{{id}}">
  ...Сохранить
  ...</button>
  ...</div>
  ...</div>
  ...`;

  ...const templateData = {
  ...  ...item,
  ...  formattedDate: formatDate(item.publishedAt || item.date),
  ...  description: truncateText(item.description, 150)
  ...};

  ...const element = createElementFromData(templateData, template);

  ...const saveButton = element.querySelector('button');
  ...saveButton.addEventListener('click', () => {
  ...  ...this.saveItem(item);
  ...});

  ...return element;
}

// Использование утилит фильтрации и сортировки
filterAndSortData(data, filters, sortBy) {
  ...let filteredData = filterData(data, filters);

  ...if (sortBy) {
  ...  ...filteredData = sortData(filteredData, sortBy.key, sortBy.direction);
  ...}

  ...return filteredData;
}
```

Часть 4. Интеграция с UI и обработка состояний

Основной модуль приложения (js/script.js):

```
import ApiService from './api/apiService.js';
import LocalStorageService from './storage/localStorage.js';
import { formatDate, truncateText } from './utils/dataParser.js';

class APIIntegrationManager {
  constructor() {
    this.localStorage = new LocalStorageService();
    this.api = null;
    this.currentData = null;
    this.init();
  }

  async init() {
    await this.initializeAPI();
    this.setupEventListeners();
    this.loadCachedData();
    this.setupSecurityMeasures();
  }

  // Инициализация API сервиса
  async initializeAPI() {
    // Выберите конфигурацию согласно вашему варианту
    const apiConfig = {
      url: 'https://api.yourservice.com/v1',
      apiKey: 'your_api_key_here', // В реальном проекте из .env
    };

    this.api = new ApiService(apiConfig.url, apiConfig.apikey);
  }

  setupEventListeners() {
    // Форма поиска
    const searchForm = document.getElementById('search-form');
    if (searchForm) {
      searchForm.addEventListener('submit', (e) => {
        e.preventDefault();
        this.handleSearch();
      });
    }

    // Кнопка обновления данных
    const refreshBtn = document.getElementById('refresh-btn');
    if (refreshBtn) {
      refreshBtn.addEventListener('click', () => {
        this.refreshData();
      });
    }
  }
}
```

```
→ // Кнопка очистки кэша
→ const clearCacheBtn = document.getElementById('clear-cache-btn');
→ if (clearCacheBtn) {
→   clearCacheBtn.addEventListener('click', () => {
→     this.clearCache();
→   });
→ }

→ // Обработка поиска
→ async handleSearch() {
→   const searchInput = document.getElementById('search-input');
→   const query = searchInput.value.trim();

→   if (!query) {
→     this.showError('Введите поисковый запрос');
→     return;
→   }

→   await this.fetchData({ q: query });
→ }

→ // Загрузка данных из API
→ async fetchData(params = {}) {
→   this.showLoading(true);

→   try {
→     // Проверяем кэш перед запросом к API
→     const cacheKey = `api_data_${JSON.stringify(params)}`;
→     const cachedData = this.localStorage.get(cacheKey, null, 3600000); // 1 час

→     if (cachedData) {
→       this.currentData = cachedData;
→       this.renderData(cachedData);
→       this.showNotification('Данные загружены из кэша');
→       return;
→     }

→     // Запрос к API
→     const data = await this.api.get('/search', params);
→     this.currentData = data;
→   }
→ }
```

```
+ // Кэшируем результаты
+ this.localStorage.set(cacheKey, data);
+ this.localStorage.set('last_api_call', new Date().toISOString());
+
+ this.renderData(data);
+ this.showNotification('Данные успешно загружены');
+ } catch (error) {
+   this.handleAPIError(error);
+ } finally {
+   this.showLoading(false);
+ }
+ }
+
+ // Загрузка кэшированных данных
+ loadCachedData() {
+   const lastData = this.localStorage.get('last_api_data');
+   if (lastData) {
+     this.currentData = lastData;
+     this.renderData(lastData);
+   }
+ }
+
+ // Обработка ошибок API
+ handleAPIError(error) {
+   console.error('API Error:', error);
+
+   let errorMessage = 'Произошла ошибка при загрузке данных';
+
+   if (error.message.includes('404')) {
+     errorMessage = 'Запрашиваемые данные не найдены';
+   } else if (error.message.includes('429')) {
+     errorMessage = 'Превышен лимит запросов. Попробуйте позже';
+   } else if (error.message.includes('401')) {
+     errorMessage = 'Ошибка авторизации. Проверьте API ключ';
+   } else if (!navigator.onLine) {
+     errorMessage = 'Отсутствует подключение к интернету';
+   }
+
+   this.showError(errorMessage);
+ }
```

```

121 // Показываем кэшированные данные если есть
122 const cachedData = this.localStorage.get('last_api_data');
123 if (cachedData) {
124   this.showNotification('Показаны кэшированные данные');
125 }
126 }
127
128 // Рендеринг данных в интерфейс
129 renderData(data) {
130   const container = document.getElementById('data-container');
131   if (!container) return;
132
133   container.innerHTML = '';
134
135   if (data && data.length > 0) {
136     data.forEach((item) => {
137       const element = this.createDataElement(item);
138       container.appendChild(element);
139     });
140   } else {
141     container.innerHTML = '<p class="no-data">данные не найдены</p>';
142   }
143 }
144
145 // Создание элемента данных
146 createDataElement(item) {
147   const element = document.createElement('div');
148   element.className = 'data-item';
149
150   // Адаптируйте под структуру данных вашего API
151   element.innerHTML = `
152     <h3 class="data-title">${item.title || item.name}</h3>
153     <p class="data-description">${item.description || ''}</p>
154     <div class="data-meta">
155       ${
156         item.date
157         ? `<span class="data-date">${formatDate(item.date)}</span>`
158         : ''
159       }
160     </div>
161     <button class="btn-save" data-id="${item.id}">Сохранить</button>
162   `;
163 }

```

```
+ // Обработчик для кнопки сохранения
+ const saveBtn = element.querySelector('.btn-save');
+ saveBtn.addEventListener('click', () => {
+   this.saveItem(item);
+ });
+
+ return element;
+ }
+
+ // Сохранение элемента
+ saveItem(item) {
+   const savedItems = this.localStorage.getItem('saved_items', []);
+   savedItems.push({
+     ...item,
+     savedAt: new Date().toISOString(),
+   });
+   this.localStorage.setItem('saved_items', savedItems);
+   this.showNotification('Элемент сохранен');
+ }
+
+ // Обновление данных
+ async refreshData() {
+   this.localStorage.clearExpired();
+   await this.fetchData();
+ }
+
+ // Очистка кэша
+ clearCache() {
+   const keys = this.localStorage.getAllKeys();
+   keys.forEach((key) => {
+     if (key !== 'app_settings' && key !== 'saved_items') {
+       this.localStorage.removeItem(key);
+     }
+   });
+   this.showNotification('Кэш очищен');
+ }
+
+ // Настройка мер безопасности
+ setupSecurityMeasures() {
+   // Очистка устаревших данных при загрузке
+   this.localStorage.clearExpired();
+ }
```

```

// Логирование API вызовов
const originalFetch = window.fetch;
window.fetch = async (...args) => {
  console.log('API Call:', args[0]);
  return originalFetch.apply(this, args);
};

// Вспомогательные методы UI
showLoading(show = true) {
  const loader = document.getElementById('loading-indicator');
  if (loader) {
    loader.style.display = show ? 'block' : 'none';
  }
}

showError(message) {
  this.showNotification(message, 'error');
}

showNotification(message, type = 'success') {
  const notification = document.createElement('div');
  notification.className = `notification notification-${type}`;
  notification.textContent = message;
  notification.style.cssText = `
    position: fixed;
    top: 20px;
    right: 20px;
    padding: 12px 20px;
    border-radius: 4px;
    color: white;
    z-index: 1000;
    background: ${type === 'error' ? '#f44336' : '#4CAF50'};
  `;
  document.body.appendChild(notification);

  setTimeout(() => {
    notification.remove();
  }, 3000);
}

// Инициализация приложения
document.addEventListener('DOMContentLoaded', () => {
  new APIIntegrationManager();
});

```

Часть 5. Реализация индивидуального задания

Шаг 4: Обработка данных и отображение результатов

- Полученные данные (например, погода, местоположение, видео и т.д.) должны быть обработаны и отображены на веб-странице.
 - Пример для отображения погоды:


```
// Пример специализированного обработчика для погодного API
class WeatherDataHandler {
  constructor(apiService) {
    this.api = apiService;
  }

  async getWeatherData(city) {
    try {
      const data = await this.api.get('/weather', { q: city, units: 'metric' });
      return this.formatWeatherData(data);
    } catch (error) {
      throw new Error(`Failed to fetch weather data: ${error.message}`);
    }
  }

  formatWeatherData(rawData) {
    return {
      temperature: Math.round(rawData.main.temp),
      description: rawData.weather[0].description,
      icon: rawData.weather[0].icon,
      humidity: rawData.main.humidity,
      windSpeed: rawData.wind.speed,
      city: rawData.name,
    };
  }

  renderWeather(weatherData, container) {
    container.innerHTML = `
      <div class="weather-card">
        <h2>Погода в ${weatherData.city}</h2>
        <div class="weather-main">
          
          <span class="temperature">${weatherData.temperature}°C</span>
        </div>
        <p class="weather-description">${weatherData.description}</p>
        <div class="weather-details">
          <span>Влажность: ${weatherData.humidity}%</span>
          <span>Ветер: ${weatherData.windSpeed} м/с</span>
        </div>
      </div>
    `;
  }
}
```

1. Добавить динамическое обновление интерфейса в зависимости от состояния данных.

Шаг 5: Обеспечение безопасности

1. **API-ключи** должны быть хранены в `.env` файле, но следует помнить, что это не защищает от утечек на клиенте.
 - API-ключи в клиентских приложениях всегда будут доступны конечному пользователю, но можно ограничить их использование:
 - Ограничьте использование API-ключей на сервере или используйте ограничения по IP.
 - В случае, если API предоставляет возможность работы с **OAuth 2.0**, настроить авторизацию через OAuth вместо хранения API-ключа.
2. В проекте можно также добавить механизм проверки безопасности, например:
 - Использование HTTPS для всех запросов.
 - Ограничение по методу HTTP, например, только GET для публичных данных.

```
// Безопасная работа с API ключами
class SecureAPIManager {
  constructor() {
    this.apiKey = this.getAPIKey();
  }

  getAPIKey() {
    // В реальном приложении ключи должны приходить с сервера
    // Для демонстрации используем переменные окружения
    return process.env.API_KEY || 'demo_key';
  }

  // Проверка CORS и безопасных методов
  async makeSecureRequest(url, options = {}) {
    const secureOptions = {
      ...options,
      mode: 'cors',
      credentials: 'same-origin',
    };

    try {
      const response = await fetch(url, secureOptions);

      if (!response.ok) {
        throw new Error(`Secure request failed: ${response.status}`);
      }

      return await response.json();
    } catch (error) {
      console.error('Secure request error:', error);
      throw error;
    }
  }
}
```

Часть 6. Тестирование и обработка ошибок

1. Тестирование различных сценариев:

```
// Модуль тестирования API
class APITester {
  static async testAPIConnection(apiService) {
    const testScenarios = [
      { name: 'Успешный запрос', params: {} },
      { name: 'Запрос с параметрами', params: { q: 'test' } },
      { name: 'Неверные параметры', params: { invalid: 'param' } },
    ];

    for (const scenario of testScenarios) {
      try {
        console.log(`Testing: ${scenario.name}`);
        const result = await apiService.get('/test', scenario.params);
        console.log(`✓ Success:`, result);
      } catch (error) {
        console.log(`X Error:`, error.message);
      }
    }
  }

  static testOfflineFunctionality(localStorageService) {
    // Тестирование работы без интернета
    const testData = { test: 'offline_data' };
    localStorageService.set('offline_test', testData);
    const retrieved = localStorageService.get('offline_test');
    console.log(
      `Offline test:`,
      retrieved?.test === 'offline_data' ? '✓ PASS' : 'X FAIL',
    );
  }
}
```

2. Протестируйте работу при отключенном интернете, в разных браузерах и проверьте, как приложение взаимодействует с выбранным API.
3. Проверьте корректность работы с хранилищем
4. Убедитесь в правильной обработке ошибок API

Часть 7. Фиксация изменений

```
git add .
git commit -m "feat: implement API integration and data storage for [тип проекта]"
git push origin feature/async-data-storage
```

5. СОДЕРЖАНИЕ ОТЧЕТА

Отчёт о выполнении лабораторной работы должен содержать:

1. **Титульный лист**
2. **Цель работы**
3. **Краткие теоретические сведения** об асинхронном JavaScript, Fetch API и клиентских хранилищах
4. **Ход работы:**
 - Описание выполненных действий
 - Скриншоты:
 - Инструменты разработчика, показывающие сетевые запросы
 - Данные в LocalStorage/SessionStorage
 - Консоль с логами асинхронных операций
 - Обработка ошибок при недоступности API
 - Примеры запросов и ответов API
 - Ссылка на коммит с выполненной работой
5. **Ответы на контрольные вопросы**
6. **Выводы** по работе

6. ТРЕБОВАНИЯ К ОФОРМЛЕНИЮ И СДАЧЕ

- Выполненное задание и отчет должны быть представлены преподавателю для защиты
- Электронная версия отчета (в формате PDF) должна быть загружена в систему СЭО
- Отчет должен быть оформлен в соответствии со Стандартом Предприятия (СТП)
- Студенты работают в тех же бригадах, что и в предыдущих работах – **Для выбора номера варианта для бригады используется [Генератор вариантов](#).**
- **1 подгруппа:** выполняет варианты **1-8**
- **2 подгруппа:** выполняет варианты **9-16**

ИНДИВИДУАЛЬНЫЕ ВАРИАНТЫ ЗАДАНИЙ

Задание для всех вариантов:

Реализуйте интеграцию с внешним API согласно вашему типу проекта. Организуйте кэширование данных и синхронизацию между локальным хранилищем и сервером.

1 ПОДГРУППА (Варианты 1-8)

- **Вариант 1 - Новостной портал "NewsHub":**
 - Интегрируйте NewsAPI, Guardian API, Currents API для загрузки актуальных новостей
 - Реализуйте кэширование новостей в LocalStorage
 - Добавьте возможность оффлайн-чтения последних загруженных новостей
- **Вариант 2 - Интернет-магазин "TechStore":**
 - Используйте FakeStore API для загрузки товаров, Shopify API, Stripe API (для платежей)
 - Реализуйте корзину с сохранением в LocalStorage
 - Добавьте синхронизацию корзины при восстановлении соединения
- **Вариант 3 - Кулинарный блог "TastyBlog":**
 - Интегрируйте TheMealDB API для рецептов, Spoonacular API, Edamam API
 - Реализуйте сохранение избранных рецептов
 - Добавьте оффлайн-доступ к избранным рецептам
- **Вариант 4 - Сайт мероприятия "EventSpace":**
 - Используйте Eventbrite API, Ticketmaster API, Meetup API
 - Реализуйте кэширование информации о мероприятиях
 - Добавьте оффлайн-просмотр расписания

- **Вариант 5 - Социальная сеть "SocialSphere":**

- Используйте JSONPlaceholder для имитации социальной сети, MockAPI, Firebase Realtime Database
- Реализуйте черновики постов с автосохранением
- Добавьте очередь отправки постов при плохом соединении

- **Вариант 6 - FAQ система "HelpDesk":**

- Создайте систему загрузки FAQ с сервера. Используйте Custom JSON API, Airtable API
- Реализуйте кэширование часто задаваемых вопросов
- Добавьте офлайн-поиск по кэшированным вопросам

- **Вариант 7 - Портфолио фотографа "PhotoPortfolio":**

- Интегрируйте Unsplash API для демонстрации работ, Pexels API, Flickr API
- Реализуйте кэширование галереи для офлайн-просмотра
- Добавьте возможность отмечать избранные фото

- **Вариант 8 - Образовательная платформа "EduPlatform":**

- Используйте образовательные API (Khan Academy API, Coursera API, Udemy API)
- Реализуйте сохранение прогресса обучения
- Добавьте синхронизацию прогресса при появлении сети

2 ПОДГРУППА (Варианты 9-16)

- **Вариант 9 - Ресторан "GourmetResto":**

- Интегрируйте API для меню и бронирования (Zomato API, Yelp API, OpenTable API)
- Реализуйте кэширование меню
- Добавьте офлайн-просмотр меню и бронирование с последующей синхронизацией

- **Вариант 10 - Библиотека "DigitalLibrary":**

- Используйте API (Google Books API, Open Library API, Goodreads API)
- Реализуйте кэширование результатов поиска
- Добавьте офлайн-библиотеку избранных книг

- **Вариант 11 - Киносайт "MovieHub":**

- Интегрируйте API для информации о фильмах (OMDb API, The Movie Database (TMDb) API)
- Реализуйте кэширование данных о фильмах
- Добавьте офлайн-доступ к избранным фильмам

- **Вариант 12 - Бронирование отелей "HotelBooking":**

- Используйте API (Amadeus API, [Booking.com](https://booking.com) API, [Hotels.com](https://hotels.com) API или аналогичный)
- Реализуйте кэширование доступных отелей
- Добавьте сохранение истории поиска

- **Вариант 13 - Музыкальный сервис "MusicStream":**

- Интегрируйте API (Spotify Web API, [Last.fm](https://last.fm) API, Deezer API) для информации о музыке
- Реализуйте кэширование плейлистов
- Добавьте офлайн-доступ к избранным трекам

- **Вариант 14 - SaaS платформа "CloudService":**

- Создайте REST API для управления данными. Используйте Custom REST API, Firebase, Supabase
- Реализуйте оптимистичные обновления интерфейса
- Добавьте обработку конфликтов при синхронизации

- **Вариант 15 - Спортивный портал "SportArena":**

- Используйте спортивные API (The Sports DB, ESPN API, [Football-Data.org](https://football-data.org))
- Реализуйте кэширование результатов матчей
- Добавьте уведомления о новых результатах

- **Вариант 16 - Научный журнал "ScienceJournal":**

- Интегрируйте API (CrossRef API, arXiv API, PubMed API) для научных статей
- Реализуйте кэширование статей для офлайн-чтения
- Добавьте систему закладок с синхронизацией

КОНТРОЛЬНЫЕ ВОПРОСЫ

Для всех вариантов вопросы одинаковые:

1. **Каковы основные различия между LocalStorage и SessionStorage?** В каких сценариях следует использовать каждый из них?
2. **Как механизм Promise улучшает работу с асинхронными операциями?** Какие преимущества предоставляют async/await?
3. **Какие стратегии обработки ошибок вы реализовали при работе с API?** Как обеспечить отказоустойчивость приложения?
4. **Какие методы кэширования данных вы применили?** Как поддерживать актуальность кэшированных данных?
5. **Как реализовать офлайн-функциональность?** Какие данные наиболее критичны для кэширования?
6. **Что такое CORS и как он влияет на взаимодействие с API?** Какие методы обхода CORS вы знаете?
7. **Каковы лучшие практики безопасности при работе с API ключами на клиенте?**
8. **Как реализовать пагинацию при работе с большими объемами данных от API?**
9. **Какие HTTP-статусы наиболее важны при работе с REST API?** Как их правильно обрабатывать?
10. **Как оптимизировать производительность при частых запросах к API?**

ПРАКТИЧЕСКИЕ РЕКОМЕНДАЦИИ

- Всегда обрабатывайте возможные ошибки при работе с API
- Используйте индикаторы загрузки для улучшения UX
- Ограничивайте размер данных в LocalStorage (обычно 5-10MB)
- Регулярно очищайте устаревшие данные из хранилища
- Тестируйте приложение в условиях плохого соединения
- Используйте AbortController для отмены запросов при необходимости
- Реализуйте retry-логику для неудачных запросов

Примечание: Созданная в этой работе система работы с данными будет использоваться в следующих лабораторных работах при интеграции с React и создании полнофункционального веб-приложения.

Добавить ответ на задание

Состояние ответа

Номер попытки	Попытка 1.
Состояние ответа на задание	Ответы на задание еще не представлены
Состояние оценивания	Не оценено
Оставшееся время	Задание просрочено на: 4 дн. 10 час.
Последнее изменение	-
Комментарии к ответу	> Комментарии (0) .